

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



MẠNG MÁY TÍNH - CO3093

BÁO CÁO BÀI TẬP LỚN 1

GVHD: Bùi Xuân Giang

Lớp: L08

Sinh viên: Hà Cao Đức Minh - 2312059

Lê Khánh Duy - 2310467

Nguyễn Tấn Phước - 2312776

Dương Đức Huy - 2311126

Ngày 18 tháng 5 năm 2026

Mục lục

1	Giới Thiệu	2
2	Tổng Quan Kiến Trúc	2
3	Hybrid Chat Application	3
3.1	Mục Tiêu	3
3.2	Các Role Của Ứng Dụng	3
3.3	Tracker Server	4
3.4	Peer Server	4
3.5	Peer Discovery	5
3.6	Direct Message	5
3.7	Broadcast Message	6
3.8	Channel Management	7
3.9	Notification	7
3.10	Web UI Cho Chat App	8
4	Authentication	8
4.1	Mục Tiêu	8
4.2	Cấu Hình Credential	9
4.3	Route Protection	9
4.4	Basic Auth Flow	10
4.5	Session Cookie	11
5	Security	11
5.1	Kiểm Tra Dữ Liệu Đầu Vào	11
5.2	JSON Parsing An Toàn	12
5.3	Chống Gửi Tin Nhắn Cho Chính Mình	12
5.4	Bảo Vệ Endpoint Nhận Tin Nhắn	12
5.5	So Sánh Password An Toàn Hơn	12
6	Kiến Trúc HTTP Server Core	13
6.1	Phân Tích Request (Request Parsing)	13
6.2	Xây Dựng Response (Response Construction)	13
6.3	HttpAdapter và Quy Trình Điều Phối	14

7	Các Cơ Chế Xử Lý Đồng Thời (Concurrency Models)	14
7.1	Mô Hình Đa Luồng (Multi-threading)	14
7.2	Mô Hình Callback (Non-blocking I/O)	14
7.3	Mô Hình Coroutine (Asynchronous I/O)	15
8	Proxy Server và Điều Phối Lưu Lượng	15
8.1	Cơ Chế Reverse Proxy	15
8.2	Chính Sách Điều Phối Round-Robin	15
8.3	Xử Lý Lỗi và Forwarding	16
9	Hướng Dẫn Test Tổng Quan	16
9.1	Test Chat Bằng UI	16
9.2	Test Bằng Curl	17
9.3	Test Proxy	17
10	Kết Luận	18
11	Mã Nguồn	18

1 Giới Thiệu

Dự án được xây dựng dựa trên source code `AsynapRous` của bài tập lớn môn Computer Networking. Mục tiêu của dự án là hoàn thiện một HTTP server đơn giản có khả năng phục vụ file tĩnh, xử lý route ứng dụng, hỗ trợ cơ chế đồng thời, proxy server, đồng thời xây dựng thêm ứng dụng Hybrid Chat App có authentication và security cơ bản.

Trong phần hiện thực, nhóm tập trung hoàn thiện các thành phần quan trọng sau:

- HTTP backend server có thể xử lý nhiều client.
- Request/response adapter để parse HTTP request và tạo HTTP response.
- Hybrid Chat App gồm Tracker server và nhiều Peer server.
- Authentication bằng Basic Auth và session cookie.
- Các kiểm tra security cơ bản cho dữ liệu đầu vào của chat app.
- Web UI để test trực tiếp bằng trình duyệt.
- Proxy server và file cấu hình proxy.

Trong đó, phần Hybrid Chat App, Authentication và Security là phần chính được mở rộng nhiều nhất so với source code ban đầu.

2 Tổng Quan Kiến Trúc

Kiến trúc dự án gồm các thành phần chính:

- `daemon/`: chứa core HTTP server, request parser, response builder, backend và proxy.
- `apps/sampleapp.py`: chứa ứng dụng mẫu và các role mới cho Hybrid Chat App.
- `www/`: chứa các file HTML phục vụ giao diện.
- `static/`: chứa CSS và image cho UI.
- `config/proxy.conf`: cấu hình định tuyến proxy.
- `scripts/test_chat_flow.py`: script test tự động luồng chat.
- `start_sampleapp.py`, `start_backend.py`, `start_proxy.py`: các file khởi động server.

Server sử dụng class `AsynapRous` để đăng ký route. Mỗi route được gán với HTTP method và path. Khi có request đến, `HttpAdapter` parse request, tìm route phù hợp, kiểm tra authentication nếu route yêu cầu, gọi handler và trả về HTTP response.

3 Hybrid Chat Application

3.1 Mục Tiêu

Hybrid Chat App được thiết kế theo mô hình kết hợp giữa Client-Server và Peer-to-Peer.

Phần Client-Server được thể hiện qua Tracker server. Tracker đóng vai trò như directory server, lưu danh sách peer đang online và cung cấp API để các peer tìm thấy nhau.

Phần Peer-to-Peer được thể hiện qua việc các message chat không đi qua Tracker. Sau khi biết IP và port của nhau, các peer gửi HTTP request trực tiếp đến endpoint của peer đích.

Cách thiết kế này giữ đúng tính chất “hybrid”:

- Tracker giúp discovery và quản lý danh sách peer.
- Peer tự giao tiếp trực tiếp với peer khác.
- Tracker không lưu nội dung chat và không forward message.

3.2 Các Role Của Ứng Dụng

Ứng dụng trong `apps/sampleapp.py` có thể chạy với ba role:

- `sample`: chạy ứng dụng REST mẫu ban đầu.
- `tracker`: chạy Tracker server.
- `peer`: chạy một peer chat riêng biệt, ví dụ Alice hoặc Bob.

Role được chọn khi chạy file `start_sampleapp.py`:

```
1 python start_sampleapp.py --role tracker --server-ip
  127.0.0.1 --server-port 8000
2 python start_sampleapp.py --role peer --username alice --
  server-ip 127.0.0.1 --server-port 9001 --tracker-ip
  127.0.0.1 --tracker-port 8000
3 python start_sampleapp.py --role peer --username bob --server
  -ip 127.0.0.1 --server-port 9002 --tracker-ip 127.0.0.1 --
  tracker-port 8000
```

Việc dùng chung một endpoint giúp không cần tạo thêm module chat riêng, đồng thời vẫn giữ được cấu trúc source code gốc.

3.3 Tracker Server

Tracker server được tạo bằng hàm `_create_tracker_app()`. Tracker lưu danh sách peer trong memory bằng dictionary `peers`.

Mỗi peer được lưu với các thông tin:

- **username**: tên peer, ví dụ `alice`, `bob`.
- **ip**: địa chỉ IP của peer.
- **port**: port peer đang lắng nghe.
- **status**: trạng thái online/offline.
- **channels**: danh sách channel peer đang tham gia.

Tracker cung cấp các endpoint chính:

Endpoint	Method	Chức năng
<code>/login</code>	POST	Đăng ký hoặc cập nhật thông tin peer cơ bản
<code>/submit-info</code>	POST	Peer gửi thông tin đầy đủ lên tracker
<code>/get-list</code>	GET	Lấy danh sách peer hiện có
<code>/logout</code>	POST	Chuyển trạng thái peer sang offline

Khi peer start, peer tự động gọi `/submit-info` để đăng ký với Tracker. Vì vậy người dùng không cần đăng ký thủ công trước khi chat.

3.4 Peer Server

Mỗi user trong chat app là một HTTP server riêng. Ví dụ:

- Alice chạy tại `127.0.0.1:9001`.
- Bob chạy tại `127.0.0.1:9002`.

Peer server được tạo bằng hàm `_create_peer_app(...)`. Mỗi peer có state riêng trong memory:

```

1 username
2 ip
3 port

```

```

4 tracker_ip
5 tracker_port
6 channels
7 messages
8 notifications
9 known_peers

```

State này chỉ tồn tại trong quá trình server đang chạy. Khi restart server, messages và notifications sẽ mất vì dự án không sử dụng database.

3.5 Peer Discovery

Peer discovery là quá trình một peer tìm thông tin peer khác thông qua Tracker. Ví dụ Alice muốn gửi tin nhắn cho Bob:

1. Alice gọi Tracker endpoint `/get-list`.
2. Tracker trả về danh sách peer online.
3. Alice tìm peer có `username = bob`.
4. Alice lấy IP và port của Bob.
5. Alice gửi HTTP request trực tiếp đến Bob.

Hàm `find_peer(target_username)` trong peer app thực hiện việc tìm peer theo username. Nếu Tracker không trả về Bob hoặc Bob offline, Alice sẽ trả về lỗi cho người dùng.

3.6 Direct Message

Direct message được xử lý qua endpoint `/direct-message` trên peer gửi. Payload mẫu:

```

1 {
2   "to": "bob",
3   "channel": "general",
4   "message": "Hello Bob"
5 }

```

Luồng xử lý:

1. Peer gửi parse JSON body.

2. Kiểm tra username người nhận hợp lệ.
3. Kiểm tra không gửi tin nhắn cho chính mình.
4. Kiểm tra message không rỗng và không vượt quá độ dài tối đa.
5. Lấy danh sách peer từ Tracker.
6. Tìm peer đích theo username.
7. Gửi POST trực tiếp đến peer đích qua endpoint `/send-peer`.
8. Peer đích lưu message với `direction = incoming`.
9. Peer gửi lưu message với `direction = outgoing`.

Endpoint `/send-peer` là endpoint nhận tin nhắn trực tiếp từ peer khác. Endpoint này cũng được bảo vệ bằng authentication.

3.7 Broadcast Message

Broadcast message được xử lý qua endpoint `/broadcast-peer`. Broadcast khác direct message ở chỗ người gửi không chỉ định một người nhận duy nhất. Peer gửi sẽ lấy danh sách tất cả peer từ Tracker, bỏ qua chính nó, rồi gửi message đến từng peer còn lại.

Luồng xử lý:

1. Peer gửi nhận message và channel.
2. Kiểm tra message hợp lệ.
3. Lấy danh sách peer từ Tracker.
4. Lặp qua từng peer online.
5. Bỏ qua peer có username trùng với peer hiện tại.
6. Gửi POST `/send-peer` đến từng peer.
7. Lưu danh sách gửi thành công vào `sent`.
8. Lưu danh sách gửi thất bại vào `failed`.

Nếu một peer bị lỗi kết nối, broadcast vẫn tiếp tục với các peer khác. Cách này giúp hệ thống không bị dừng hoàn toàn khi một peer offline hoặc không phản hồi.

3.8 Channel Management

Mỗi peer có danh sách channel riêng. Mặc định khi peer được tạo, channel ban đầu là:

```
1 ["general"]
```

Peer có thể tham gia channel mới qua endpoint `/join-channel`. Payload mẫu:

```
1 {  
2   "channel": "study"  
3 }
```

Khi peer join channel, channel mới được thêm vào state local của peer và peer cập nhật lại thông tin với Tracker. Channel là local theo từng peer, vì vậy nếu Alice join `study` thì Bob không tự động join `study`. Bob chỉ thấy hoặc có channel đó khi Bob tự join, hoặc khi Bob nhận message thuộc channel đó và server tự thêm channel vào danh sách local.

Messages có trường `channel`, vì vậy UI có thể lọc message theo channel hiện tại thông qua:

```
1 GET /messages?channel=general
```

3.9 Notification

Khi peer nhận message qua `/send-peer`, server tạo notification trong state local. Notification có dạng:

```
1 {  
2   "type": "new_message",  
3   "from": "alice",  
4   "channel": "general",  
5   "message": "Hello",  
6   "timestamp": "..."  
7 }
```

Các endpoint notification:

Endpoint	Method	Chức năng
/notifications	GET	Lấy danh sách notification
/notifications/clear	POST	Xóa tất cả notification

Tính năng này giúp UI hiển thị tin nhắn mới đến mà không cần lưu vào database.

3.10 Web UI Cho Chat App

Giao diện chat nằm ở `www/chat.html`, style nằm ở `static/css/styles.css`. UI hỗ trợ các thao tác:

- Xem thông tin peer hiện tại.
- Xem danh sách channel.
- Join channel mới.
- Xem danh sách peer online.
- Gửi direct message.
- Broadcast message.
- Xem message theo channel.
- Xem và clear notification.

UI gọi API của peer server bằng JavaScript `fetch`. Khi mở trang, UI hỏi username và password Basic Auth. Thông tin này được lưu trong `sessionStorage` của browser để phục vụ các request tiếp theo trong cùng tab.

Danh sách peer trên UI đã lọc bỏ chính peer hiện tại. Ví dụ khi đang ở UI của Alice, danh sách peer chỉ hiện Bob. Điều này tránh việc người dùng nhầm lẫn và gửi message cho chính mình.

4 Authentication

4.1 Mục Tiêu

Authentication được thêm vào để bảo vệ các API thao tác chat. Nếu không có authentication, bất kỳ client nào biết IP và port của peer cũng có thể đọc messages, gửi message hoặc xóa notifications. Vì vậy các endpoint nhạy cảm của peer cần được bảo vệ.

Dự án hiện thực Basic Auth kết hợp session cookie:

- Lần đầu client gửi `Authorization: Basic ....`
- Server kiểm tra username/password.
- Nếu hợp lệ, server tạo session token và trả về `Set-Cookie`.
- Các request sau có thể dùng cookie session thay cho việc gửi lại Basic Auth.

4.2 Cấu Hình Credential

Credential mặc định:

```
1 username: admin
2 password: password
```

Có thể đổi credential khi chạy peer:

```
1 python start_sampleapp.py --role peer --username alice --
    server-ip 127.0.0.1 --server-port 9001 --tracker-ip
    127.0.0.1 --tracker-port 8000 --auth-user admin --auth-
    password password
```

Hoặc dùng biến môi trường:

```
1 $env:CHAT_AUTH_USER="admin"
2 $env:CHAT_AUTH_PASSWORD="password"
```

Trong demo, các peer nên dùng cùng credential để peer gửi message trực tiếp cho nhau thành công, vì endpoint `/send-peer` của peer đích cũng yêu cầu auth.

4.3 Route Protection

Class `AsynapRous` được mở rộng để route có thể khai báo `auth=True`. Ví dụ:

```
1 @peer_app.route("/direct-message", methods=["POST"], auth=
    True)
2 def direct_message(...):
3     ...
```

Khi route có `_auth_required = True`, `HttpAdapter` sẽ gọi `_authorize_request(req)` trước khi dispatch handler.

Các endpoint peer được bảo vệ:

- /send-peer
- /direct-message
- /broadcast-peer
- /connect-peer
- /messages
- /channels
- /notifications
- /notifications/clear
- /join-channel
- /peer-info
- /peers

Tracker không bắt buộc Basic Auth trong bản hiện tại, vì Tracker chỉ lưu thông tin peer online và phục vụ discovery.

4.4 Basic Auth Flow

Khi request đến protected route:

1. Server kiểm tra session cookie `ASYNAPROUS_SESSION`.
2. Nếu cookie hợp lệ, request được cho qua.
3. Nếu chưa có cookie, server đọc header `Authorization`.
4. Header Basic Auth được decode Base64 thành `username:password`.
5. Server so sánh password với credential đã cấu hình.
6. Nếu sai, server trả về HTTP 401 `Unauthorized`.
7. Nếu đúng, server tạo session token và trả về header `Set-Cookie`.

Khi authentication thất bại, response có header:

```
WWW-Authenticate: Basic realm="Hybrid_Chat_Peer"
```

Header này cho client biết endpoint cần Basic Auth.

4.5 Session Cookie

Session cookie được tạo bởi hàm `create_session(username)` trong `daemon/utils.py`. Token được tạo bằng:

```
1 secrets.token_urlsafe(24)
```

Cookie được trả về với các thuộc tính:

```
1 Path=/
2 HttpOnly
3 SameSite=Lax
```

Ý nghĩa:

- `Path=/:` cookie được gửi cho các request trong cùng server.
- `HttpOnly:` JavaScript không đọc trực tiếp được cookie này, giảm rủi ro khi có XSS.
- `SameSite=Lax:` giảm một phần rủi ro CSRF trong các request cross-site thông thường.

Session được lưu trong memory, vì vậy khi server restart thì session cũ không còn hợp lệ.

5 Security

5.1 Kiểm Tra Dữ Liệu Đầu Vào

Chat app có các hàm validate riêng:

- `_valid_username(username)`
- `_valid_channel(channel)`
- `_valid_message(message)`

Username và channel chỉ cho phép các ký tự:

```
1 A-Z, a-z, 0-9, _, ., -
```

Độ dài username và channel tối đa là 32 ký tự. Điều này giúp tránh các giá trị bất thường, quá dài hoặc chứa ký tự đặc biệt không cần thiết.

Message phải là chuỗi không rỗng và độ dài không vượt quá `MAX_MESSAGE_LENGTH`

= 1000. Giới hạn này giúp tránh request body quá lớn và tránh việc một peer gửi message quá dài làm ảnh hưởng memory.

5.2 JSON Parsing An Toàn

Hàm `_json_body(body)` được dùng để parse body thành JSON. Nếu body không phải JSON hợp lệ, handler không crash mà trả về:

```
1 {
2   "ok": false,
3   "error": "Invalid JSON"
4 }
```

Cách xử lý này giúp server ổn định hơn khi client gửi request sai format.

5.3 Chống Gửi Tin Nhắn Cho Chính Mình

Trong endpoint `/direct-message`, server kiểm tra:

```
1 receiver == state["username"]
```

Nếu người nhận trùng với peer hiện tại, server trả về lỗi:

```
1 Cannot send direct message to self
```

Bên cạnh đó, UI cũng lọc bỏ peer hiện tại khỏi danh sách peer online. Như vậy logic được bảo vệ ở cả frontend và backend.

5.4 Bảo Vệ Endpoint Nhận Tin Nhắn

Endpoint `/send-peer` là endpoint quan trọng vì đây là nơi peer nhận message trực tiếp từ peer khác. Nếu endpoint này không có auth, bất kỳ client nào cũng có thể giả lập peer và chèn message vào state.

Việc đặt `auth=True` cho `/send-peer` giúp chỉ các client có credential hợp lệ mới gửi được message vào peer.

5.5 So Sánh Password An Toàn Hơn

Hàm `verify_basic_auth` sử dụng:

```
1 secrets.compare_digest(...)
```

So với so sánh chuỗi thông thường, cách này giảm rủi ro timing attack ở mức cơ bản khi so sánh password.

6 Kiến Trúc HTTP Server Core

Hệ thống HTTP Server được thiết kế theo nguyên tắc module hóa, tách biệt rõ ràng các khâu xử lý giao thức.

6.1 Phân Tích Request (Request Parsing)

Lớp Request trong `daemon/request.py` là thành phần đầu tiên tiếp nhận dữ liệu. Quy trình parse được thực hiện qua các bước:

- **Request Line:** Sử dụng `splitlines()` và `urlsplit` để trích xuất phương thức (GET, POST, ...), đường dẫn và phiên bản HTTP.
- **Headers:** Các header được chuyển đổi thành dictionary với key ở dạng lowercase để hỗ trợ truy xuất không phân biệt hoa thường.
- **Body Extraction:** Dựa vào header `Content-Length` để đảm bảo đọc đúng và đủ lượng dữ liệu từ socket, tránh hiện tượng mất dữ liệu hoặc treo kết nối.
- **JSON & Cookies:** Tích hợp sẵn bộ giải mã JSON cho các API hiện đại và bộ parse cookie để quản lý trạng thái phiên làm việc (session).

6.2 Xây Dựng Response (Response Construction)

Lớp Response (`daemon/response.py`) quản lý việc tạo ra các gói tin phản hồi đúng tiêu chuẩn:

- **MIME Type Detection:** Sử dụng module `mimetypes` để tự động xác định loại nội dung (text/html, image/png, application/json) dựa trên phần mở rộng của file.
- **Dynamic Header Generation:** Tự động tính toán các header quan trọng như `Content-Length`, `Date` (theo định dạng GMT), và `Server`.
- **Cookie Handling:** Hỗ trợ phương thức `set_cookie` để gửi các chỉ thị `Set-Cookie` về client với các thuộc tính bảo mật như `HttpOnly` và `SameSite`.

6.3 HttpAdapter và Quy Trình Điều Phối

HttpAdapter đóng vai trò là "bộ não" điều phối toàn bộ vòng đời của một request. Nó liên kết socket thô với logic ứng dụng:

- **Stream Reading:** Thực hiện việc đọc dữ liệu phân đoạn từ socket, đảm bảo tách biệt rõ ràng giữa header (`\r\n\r\n`) và body.
- **Route Dispatching:** Đối chiếu Method và Path với danh sách các route đã đăng ký (App Hooks). Nếu không khớp, server tự động chuyển sang chế độ phục vụ file tĩnh (Static File Serving).
- **Dependency Injection:** Sử dụng cơ chế `inspect.signature` để tự động truyền các tham số (body, query, headers, `auth_user`) vào hàm xử lý của ứng dụng một cách linh hoạt.

7 Các Cơ Chế Xử Lý Đồng Thời (Concurrency Models)

Server hỗ trợ ba mô hình xử lý đa kết nối, cho phép linh hoạt lựa chọn tùy theo môi trường triển khai.

7.1 Mô Hình Đa Luồng (Multi-threading)

Đây là mô hình mặc định trong `daemon/backend.py`. Với mỗi kết nối `accept()` thành công, server khởi tạo một `threading.Thread` mới ở chế độ `daemon`.

- **Ưu điểm:** Dễ triển khai, code tuần tự dễ hiểu, tận dụng tốt các thư viện Python đồng bộ.
- **Hạn chế:** Tốn tài nguyên bộ nhớ khi số lượng kết nối quá lớn do mỗi thread có stack riêng.

7.2 Mô Hình Callback (Non-blocking I/O)

Sử dụng module `selectors` để triển khai Event Loop thủ công:

- Server socket được đặt ở chế độ `setblocking(False)`.
- Sử dụng `DefaultSelector` để đăng ký các sự kiện `EVENT_READ`.
- Khi có sự kiện, hàm `accept_callback` hoặc `read_callback` sẽ được gọi.

- Cơ chế này cho phép một thread duy nhất quản lý hàng trăm kết nối mà không bị block.

7.3 Mô Hình Coroutine (Asynchronous I/O)

Sử dụng thư viện `asyncio` với các từ khóa `async/await`:

- **Async Server:** Khởi tạo bằng `asyncio.start_server`, tận dụng khả năng xử lý bất đồng bộ ở mức kernel.
- **Awaitable Handlers:** Cho phép các route hook của Chat App có thể là các hàm `async def`, giúp thực hiện các thao tác I/O (như gọi sang peer khác) mà không làm nghẽn server.
- **Stream Handling:** Sử dụng `StreamReader` và `StreamWriter` thay cho socket thô để quản lý luồng dữ liệu mượt mà hơn.

8 Proxy Server và Điều Phối Lưu Lượng

8.1 Cơ Chế Reverse Proxy

Proxy server (`daemon/proxy.py`) hoạt động như một cổng vào duy nhất, bảo vệ và che dấu các backend server bên trong. Nó thực hiện:

- **Hostname Resolution:** Phân tích header `Host` để quyết định backend nào sẽ xử lý request.
- **Request Forwarding:** Tạo một socket client mới để kết nối tới backend, chuyển tiếp toàn bộ payload thô và trả lại kết quả cho người dùng cuối.

8.2 Chính Sách Điều Phối Round-Robin

Để hỗ trợ khả năng mở rộng (scalability), proxy tích hợp thuật toán Round-Robin:

- **Load Balancing:** Khi một Host trỏ tới nhiều IP backend, proxy sẽ luân phiên gửi request đến từng server theo thứ tự.
- **Thread-safety:** Sử dụng `threading.Lock (RR_LOCK)` để bảo vệ biến chỉ mục `RR_STATE`, đảm bảo việc điều phối chính xác ngay cả khi có hàng ngàn request đồng thời.

8.3 Xử Lý Lỗi và Forwarding

Proxy được thiết kế để xử lý các tình huống ngoại lệ của backend:

- **502 Bad Gateway:** Trả về khi proxy không thể kết nối tới backend hoặc backend bị sập.
- **404 Not Found:** Trả về khi Host header không khớp với bất kỳ quy tắc nào trong `proxy.conf`.
- **Transparency:** Proxy cố gắng giữ nguyên các header gốc từ client (trừ khi có cấu hình `proxy_set_header`) để backend có đầy đủ thông tin về request ban đầu.

9 Hướng Dẫn Test Tổng Quan

9.1 Test Chat Bằng UI

Chạy Tracker:

```
1 python start_sampleapp.py --role tracker --server-ip  
127.0.0.1 --server-port 8000
```

Chạy Alice:

```
1 python start_sampleapp.py --role peer --username alice --  
server-ip 127.0.0.1 --server-port 9001 --tracker-ip  
127.0.0.1 --tracker-port 8000 --auth-user admin --auth-  
password password
```

Chạy Bob:

```
1 python start_sampleapp.py --role peer --username bob --server  
-ip 127.0.0.1 --server-port 9002 --tracker-ip 127.0.0.1 --  
tracker-port 8000 --auth-user admin --auth-password  
password
```

Mở UI:

```
1 http://127.0.0.1:9001/chat.html  
2 http://127.0.0.1:9002/chat.html
```

Thông tin đăng nhập UI:

```
1 admin
2 password
```

Các tính năng cần test:

- Alice thấy Bob trong danh sách peer.
- Bob thấy Alice trong danh sách peer.
- Alice gửi direct message cho Bob.
- Bob nhận message và có notification.
- Bob clear notification.
- Alice broadcast message.
- Alice join channel mới.
- Bob join hoặc nhận message theo channel mới.

9.2 Test Bằng Curl

Lấy danh sách peer:

```
1 curl.exe -i http://127.0.0.1:8000/get-list
```

Gửi direct message:

```
1 curl.exe -i -u admin:password -X POST http://127.0.0.1:9001/
  direct-message -H "Content-Type: application/json" -d "{\
  to\": \"bob\", \"channel\": \"general\", \"message\": \"Hello
  Bob\"}"
```

Lấy message của Bob:

```
1 curl.exe -i -u admin:password http://127.0.0.1:9002/messages
```

9.3 Test Proxy

Khi test proxy trên máy cá nhân, cần dùng IP thiết bị hiện tại trong `config/proxy.conf` và trong lệnh chạy server.

Ví dụ:

```
1 python start_backend.py --server-ip 10.158.117.115 --server-  
    port 9000 --mode threading  
2 python start_proxy.py --server-ip 10.158.117.115 --server-  
    port 8080  
3 curl.exe -i -H "Host: 10.158.117.115:8080" http  
    ://10.158.117.115:8080/index.html
```

10 Kết Luận

Dự án đã mở rộng source code AsynapRous ban đầu thành một hệ thống HTTP server có thể chạy ứng dụng chat hybrid. Phần Tracker đảm nhiệm vai trò discovery theo mô hình Client-Server, trong khi các Peer giao tiếp trực tiếp với nhau theo mô hình Peer-to-Peer. Hệ thống đã có các tính năng chat cơ bản như direct message, broadcast, channel, notification và UI web để test trực quan.

Phần authentication và security được bổ sung để bảo vệ các endpoint quan trọng của peer. Server hỗ trợ Basic Auth, session cookie, validate username/channel/message, parse JSON an toàn và ngăn việc gửi direct message cho chính mình. Các tính năng này đáp ứng mức demo của bài tập lớn và tạo nên nền tảng rõ ràng nếu muốn phát triển thêm thành ứng dụng chat hoàn chỉnh hơn.

11 Mã Nguồn

Link Github: <https://github.com/Phuoc205/Computer-Network---AS1>